

Research: 3D Perception Primitives (LiDAR / BEV / Multi-Modal Fusion)

The development of robust autonomous driving pipelines requires highly deterministic, hardware-agnostic implementations of core 3D perception operations. Within deep learning architectures for autonomous vehicles, perception models rely heavily on discretizing continuous spaces, projecting multi-modal inputs across disparate coordinate frames, and verifying geometric constraints.¹ As perception systems transition towards end-to-end differentiable architectures or complex fusion networks, maintaining a library of pure-function primitives—operations that map inputs to outputs without relying on global state, GPU-specific custom kernels (such as CUDA-based sparse convolutions), or mutable class states—becomes a critical engineering objective.

The transition from stateful, hardware-accelerated computer vision pipelines to pure, functional operations is heavily driven by the necessity for rigorous testing, continuous integration, and seamless deployment across heterogeneous computing environments. Standard libraries such as OpenCV or CUDA-dependent frameworks introduce opaque internal states, memory management overhead, and non-deterministic behaviors that complicate the compilation of static computational graphs. The *scion*-atoms-dl repository mandates the synthesis of perception algorithms into isolated, mathematically pure functions bounded strictly by NumPy array operations. This architectural constraint ensures that operations like voxelization, geometric projection, and rasterization can be tested without specialized GPU hardware while remaining mathematically equivalent to their highly accelerated production counterparts.

This research report investigates best-in-class, pure-function implementations for four fundamental 3D perception stages: point cloud voxelization, camera-to-LiDAR multi-modal fusion (point painting), bird's-eye-view (BEV) rasterization, and spatial verification via RANSAC-based homography estimation. The primary goal is the specification of pure Python and NumPy functions that fulfill the rigorous demands of deep learning architectures, specifically targeting data assembly and geometry operations within autonomous driving competition environments.

1. Point Cloud Voxelization: Discretizing Continuous 3D Spaces

LiDAR sensors capture the environment as continuous, sparse, and un-ordered sets of points in three-dimensional space. Deep learning models, particularly 3D Convolutional Neural Networks (CNNs) and transformer architectures, typically require structured, dense, or semi-dense grid representations to leverage standard convolution or self-attention mechanisms efficiently. Voxelization is the mathematical operation of partitioning the 3D space into a regular grid of cubic or rectangular volumes (voxels) and aggregating the raw point features that fall within

each boundary.³ The continuous nature of raw point clouds presents a structural challenge for neural networks, which inherently expect fixed-size, ordered input tensors.

1.1 Mathematical Formulation and Algorithmic Complexity

Given a point cloud dataset represented as a two-dimensional matrix where each row corresponds to a point and each column corresponds to a feature, the spatial coordinates must be translated into discrete indices. Let the point cloud be $P \in \mathbb{R}^{N \times D}$, where N is the number of points and D is the feature dimension (typically consisting of x, y, z coordinates, followed by reflectance intensity and relative timestamp). Voxelization requires mapping each continuous coordinate (x_i, y_i, z_i) to a discrete grid index $(c_{x,i}, c_{y,i}, c_{z,i})$.

The grid is defined by a 3D bounding volume, often referred to as the point cloud range

$R = (x_{min}, y_{min}, z_{min}, x_{max}, y_{max}, z_{max})$, and a strictly positive voxel size

$V = (v_x, v_y, v_z)$.⁴ The conversion from continuous space to discrete coordinates is achieved via a floor division operation applied element-wise across the spatial dimensions:

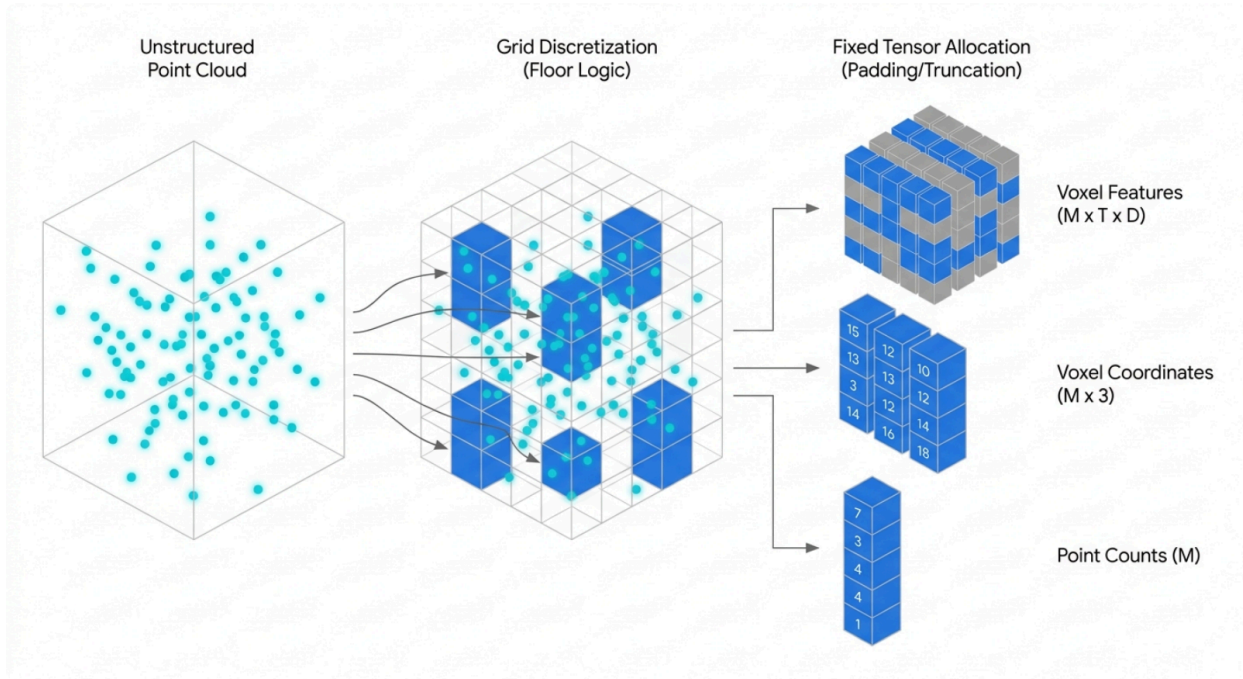
$$c_{x,i} = \left\lfloor \frac{x_i - x_{min}}{v_x} \right\rfloor, \quad c_{y,i} = \left\lfloor \frac{y_i - y_{min}}{v_y} \right\rfloor, \quad c_{z,i} = \left\lfloor \frac{z_i - z_{min}}{v_z} \right\rfloor$$

The overall grid dimensions, representing the maximum number of voxels along each spatial

axis, are computed as $G_{dim} = \lfloor (R_{max} - R_{min})/V \rfloor$. Any point yielding an index outside

the domain $0 \leq c_d < G_d$ is discarded as an out-of-bounds outlier.⁴ This spatial clipping is crucial for limiting the memory footprint of the resulting tensors, as autonomous driving datasets often contain points extending hundreds of meters, whereas the neural network may only be trained to process a specific region of interest (e.g., a 100-meter radius around the ego-vehicle).

Continuous Point Cloud Discretization to Sparse Voxel Tensors



The voxelization process computes integer grid coordinates for each continuous LiDAR point. Points within the defined spatial range are assigned to fixed-capacity voxel bins, yielding structured feature and coordinate arrays.

Once spatial indices are computed, the continuous point features must be aggregated into the discrete voxel bins. Deep learning frameworks like mmdetection3d and OpenPCDet commonly utilize "hard voxelization," which imposes strict upper limits on both the total number of populated voxels (M_{max}) and the maximum number of points permitted within any single voxel (T_{max}).⁴ This configuration results in a fixed-size tensor output of shape (M_{max}, T_{max}, D) , a property critical for static graph compilation in machine learning accelerators.

If a voxel contains fewer than T_{max} points, the remaining entries in the tensor slice are zero-padded. Conversely, if the point density within a voxel exceeds T_{max} , the algorithm must subsample or truncate the points. To maintain a pure function without reliance on a global random number generator, standard implementations simply truncate the array, retaining the first T_{max} points encountered in the array memory layout.

1.2 Pure NumPy Implementation Strategy

In conventional production systems, voxelization is accelerated using custom CUDA kernels (such as the spconv library) to perform rapid parallel scatter and gather operations directly on the GPU hardware.⁵ However, to meet the goal of a pure-function, hardware-agnostic implementation, the algorithm must be replicated entirely within the constraints of NumPy vectorization.⁷

The naive approach involves iterating over the point cloud using standard Python for loops, indexing a dictionary or multi-dimensional array based on the calculated spatial coordinates. Due to the Global Interpreter Lock (GIL) and the overhead of Python bytecode execution, this loop-based approach is computationally prohibitive for point clouds containing hundreds of thousands of observations. A highly optimized NumPy implementation circumvents explicit looping by leveraging array-based boolean masking, spatial hashing, and advanced indexing techniques.⁴

The pure-function strategy executes through several distinct phases. First, coordinate transformation occurs. The algorithm subtracts the origin vector R_{min} from all spatial point coordinates and divides by the specified voxel sizes V . Applying the np.floor function and casting the result to an integer data type yields an $N \times 3$ array of integer grid coordinates.⁴ Following the transformation, the data must be aggressively filtered. The algorithm constructs a boolean mask verifying that every coordinate falls within the bounds: $0 \leq c_{x,i} < G_x$, $0 \leq c_{y,i} < G_y$, and $0 \leq c_{z,i} < G_z$. Applying this mask to both the original point array and the coordinate array eliminates out-of-bounds observations efficiently.⁴

The most computationally complex step in a pure NumPy implementation is identifying unique voxels without iterating. To achieve this, a unique one-dimensional hash is generated for each 3D coordinate. Given the calculated grid dimensions (G_x, G_y, G_z) , the scalar hash for a point is computed as $h = c_{z,i} \cdot (G_y \cdot G_x) + c_{y,i} \cdot G_x + c_{x,i}$. This hashing flattens the 3D grid space into a 1D sequence.

Using the np.unique function with parameters return_inverse=True and return_counts=True, the algorithm identifies the populated voxels, their corresponding counts, and the mapping required to group points sharing the same hash.⁷ To enforce the M_{max} constraint, the array of unique hashes is truncated. Subsequently, a pre-allocated zeros array of shape $(M_{\{max\}}, T_{\{max\}}, D)$ is initialized. By utilizing np.argsort on the inverse indices, the point array is ordered such that points belonging to the same voxel become contiguous in memory. The algorithm then utilizes advanced integer array indexing to slice up to T_{max} points per unique voxel index, assigning them directly into the pre-allocated tensor.⁷

This pure NumPy methodology ensures deterministic outputs, predictable memory allocation,

and entirely eliminates GPU dependencies, fulfilling the repository constraints.⁷ The mathematical rigidity of this approach prevents state bleed across sequential function calls, a common source of non-determinism in autonomous driving competition environments.

1.3 Candidate Atom Specification: Voxelization

Name: `voxelize_point_cloud`

Description: Discretize a continuous 3D point cloud into a fixed-resolution voxel grid, aggregating point features within each occupied voxel into bounded dense tensors.

Source: <https://github.com/danielhavir/voxelize3d>

License: MIT

Concept type: `data_assembly`

Signature: `(points: NDArray, voxel_size: tuple[float, float, float], point_range: tuple[float,...], max_points_per_voxel: int) -> tuple`

Pure function boundary: Continuous point cloud array and explicit grid parameters in, fixed-dimension voxel arrays out. No internal CUDA kernels, no file I/O, no mutable state, and no global RNG initialization.

Contracts:

- require: `points.ndim == 2` and `points.shape >= 3`
- require: all `voxel_size` values `> 0`
- require: `len(point_range) == 6` and `point_range[i] < point_range[i+3]` for `i` in `0,1,2`
- require: `max_points_per_voxel > 0`
- ensure: `voxel_coors` are strictly within the implied grid dimensions

Witness: A synthetic point cloud (20 points) intentionally distributed across 4 known grid cells. Verify correct grid cell assignment, enforcement of `max_points_per_voxel` truncation, and accurate feature aggregation in the output tensors.

Dependencies: `numpy`

CDG stages covered: `lyft_3d_object_detection_1st/voxelization`

2. Multi-Modal Fusion: Point Painting and Camera-to-LiDAR Projection

Autonomous driving datasets, such as the nuScenes and Waymo Open Datasets, capture the environment simultaneously utilizing high-density LiDAR sensors and arrays of calibrated cameras.⁸ "PointPainting," a methodology originally introduced by Vora et al., represents a highly effective sequential multi-modal fusion architecture.¹ The technique projects semantic information derived from 2D camera images—such as pixel-wise object segmentation scores or bounding box classifications—directly onto the 3D LiDAR point cloud. The resulting output is a "painted" point cloud wherein each spatial point's feature vector is augmented with

corresponding camera-based pixel features, effectively bridging the spatial accuracy of LiDAR with the rich semantic texture of visual imagery.¹¹

2.1 Geometric Projection Mechanics

The mathematical core of the PointPainting paradigm is the strict geometric projection from the 3D LiDAR coordinate frame onto the 2D image plane.² This transformation requires the precise application of calibration matrices. The extrinsic calibration matrix (E) defines the rigid body transformation (comprising rotation and translation) between the LiDAR sensor's origin and the camera's origin. Simultaneously, the intrinsic calibration matrix (K) defines the internal pinhole model properties of the camera, primarily the focal lengths along the axes and the optical center offset.

Matrix Designation	Dimensionality	Functionality
Point Cloud (P_L)	$N \times 4$	Homogeneous representation of 3D spatial points $(X, Y, Z, 1)$.
Extrinsic (E)	4×4	Rigid transformation (Rotation R and Translation t) to Camera Frame.
Intrinsic (K)	3×3	Pinhole projection onto the 2D image plane.
Image Features (I)	$C \times H \times W$	Deep learning feature maps or RGB values extracted from the camera.

Given a 3D point represented in the LiDAR coordinate frame as $P_L = (X, Y, Z, 1)^T$ using homogeneous coordinates, the transformation to the camera coordinate frame P_C is

computed via matrix multiplication:

$$P_C = E \cdot P_L = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}$$

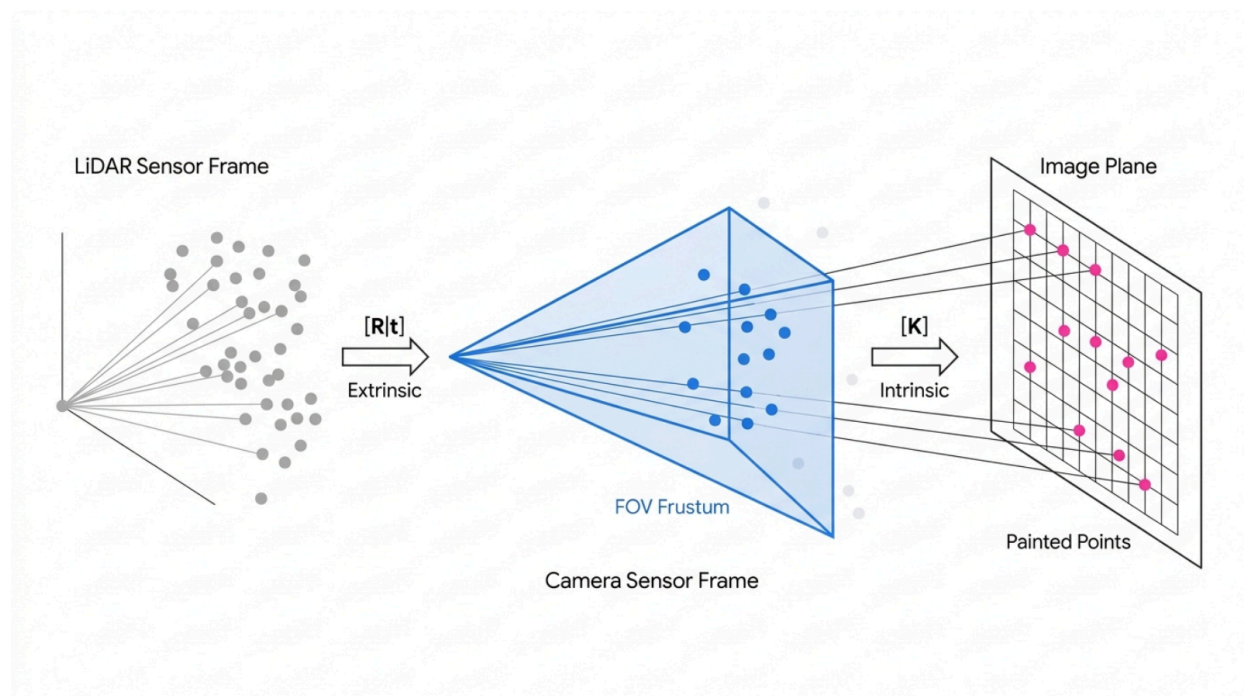
Once mapped into the camera frame, the spatial coordinates must be projected onto the 2D image plane using the 3×3 intrinsic matrix K . To accomplish this strictly using vectorized matrix multiplication in NumPy, K is typically padded to a 3×4 or 4×4 matrix K_{pad} to align with the homogeneous point representation.² The resulting vector contains the unnormalized image coordinates (u', v', w') :

$$\begin{bmatrix} u' \\ v' \\ w' \end{bmatrix} = K_{pad} \cdot \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}$$

The final pixel coordinates (u, v) on the discrete image plane are obtained through perspective division. This operation divides the horizontal and vertical components by the depth component w' (which corresponds directly to Z_c , the physical distance of the point in front of the camera, in a standard pinhole camera model):

$$u = \frac{u'}{w'}, \quad v = \frac{v'}{w'}$$

Camera-to-LiDAR Projection for Point Painting



Points in the LiDAR coordinate frame are transformed into the camera frame via the extrinsic matrix. The intrinsic matrix projects these points onto the 2D image plane, followed by perspective division and depth filtering to discard points behind the camera or outside the field of view.

2.2 Point Filtering and Feature Sampling

Projection logic alone is insufficient for a robust multi-modal fusion pipeline. Because the LiDAR sensor typically provides a 360-degree environmental scan, a substantial portion of the point cloud naturally falls outside the specific camera's limited field of view (FOV). Points must be rigorously filtered to prevent sampling out-of-bounds or physically impossible image features. A depth filter is applied to ensure points are physically located in front of the camera plane;

points with $Z_c \leq 0$ (or a minimum threshold like $Z_c < 0.1$ meters) are immediately discarded or masked.² Following depth filtration, a spatial bounds check ensures the projected pixel coordinates u and v lie strictly within the defined image dimensions: $0 \leq u < W$ and $0 \leq v < H$.²

Once the valid (u, v) coordinates are established, semantic features must be sampled from the corresponding image_features array (e.g., a multi-channel segmentation mask or RGB tensor of shape $C \times H \times W$). Because the projected coordinates u and v are continuous

floating-point values mathematically, and the image grid is inherently discrete, nearest-neighbor sampling can induce severe aliasing artifacts. Bilinear interpolation serves as the standard mathematical method for continuous sampling in computer vision pipelines.¹³

A pure NumPy implementation of bilinear interpolation completely avoids iterative loops by extracting the four integer coordinate neighbors for every valid floating-point pixel coordinate simultaneously.¹⁴ The mathematical steps map directly to vector operations. First, the algorithm computes the floor coordinates $x_0 = \lfloor u \rfloor$ and $y_0 = \lfloor v \rfloor$. Next, it computes the ceiling coordinates $x_1 = x_0 + 1$ and $y_1 = y_0 + 1$.

To prevent index out-of-bounds errors when a point projects exactly onto the extreme boundary of the image, these coordinates are strictly clamped to the image dimensions using `np.clip`.¹⁴ The algorithm then extracts the four corner pixel feature vectors from the image tensor: I_a at (y_0, x_0) , I_b at (y_1, x_0) , I_c at (y_0, x_1) , and I_d at (y_1, x_1) .

Fractional spatial weights are calculated based on the distance of the true floating-point coordinate from each integer neighbor:

- $w_a = (x_1 - u)(y_1 - v)$
- $w_b = (x_1 - u)(v - y_0)$
- $w_c = (u - x_0)(y_1 - v)$
- $w_d = (u - x_0)(v - y_0)$

The final interpolated feature vector for each point is derived by computing the weighted sum of these four corners: $F = w_a I_a + w_b I_b + w_c I_c + w_d I_d$.¹⁴ Utilizing NumPy's broadcasting capabilities, this operation executes rapidly, seamlessly decorating thousands of spatial points with multi-dimensional image features simultaneously.

2.3 Candidate Atom Specification: Point Painting Projection

Name: `project_image_to_points`

Description: Projects a 2D image feature map onto a 3D LiDAR point cloud using rigid body transforms and camera intrinsics, applying pure-numpy bilinear interpolation to sample features continuously.

Source:

https://github.com/nutonomy/nuscenes-devkit/blob/master/python-sdk/nuscenes/geometry_utils.py

License: Apache-2.0

Concept type: geometry

Signature: (points_3d: NDArray, image_features: NDArray, intrinsic: NDArray, extrinsic: NDArray, image_shape: tuple) -> NDArray

Pure function boundary: Extrinsic and intrinsic matrices and 3D geometries in; painted point array out. Relies strictly on NumPy matrix multiplication and advanced indexing. No external dependencies, mutable objects, or state caching.

Contracts:

- require: points_3d.ndim == 2 and points_3d.shape >= 3
- require: intrinsic.shape == (3, 3) or (3, 4)
- require: extrinsic.shape == (4, 4)
- require: image_features.ndim == 3
- ensure: output.shape == points_3d.shape

Witness: A synthetic 3D bounding box corner representation mapped via a known camera matrix onto a checkerboard image tensor. Verify that the appended feature values match the expected bilinear combinations of the checkerboard grid at specific depths.

Dependencies: numpy

CDG stages covered: lyft_3d_object_detection_1st/multi_modal_fusion

3. Bird's-Eye-View (BEV) Rasterization

In motion prediction, trajectory planning, and environment understanding tasks, complex, unstructured spatial data—such as vector map elements (lane boundaries, traffic intersections, crosswalks) and historical agent poses—must be fused into a unified representational space. The Bird's-Eye-View (BEV) rasterization process addresses this by converting continuous geometric polyline and polygon data into a multi-channel, top-down discrete image grid.¹⁶ This spatial alignment mechanism allows standard 2D Convolutional Neural Networks to process highly complex semantic topologies directly without requiring computationally expensive graph neural networks.¹⁸

3.1 Mapping Vector Geometry to BEV Tensors

Libraries tailored for autonomous driving, such as L5Kit and the nuScenes Devkit, approach BEV generation by instantiating distinct semantic channels within a singular tensor.¹⁶ For example, a target tensor of shape (C, H, W) might allocate channel C_0 specifically for the ego-vehicle's historical trajectory, C_1 for surrounding dynamic agent locations, C_2 for static drivable surfaces, and C_3 for explicit lane divider lines.¹⁹

The rasterization procedure initially establishes a BEV "patch." This patch acts as the computational canvas and is defined by several physical parameters: an ego-centric origin point (x_{ego}, y_{ego}) , a defined raster size in discrete pixels (H, W) , and a spatial resolution

denoting the physical distance each pixel represents (e.g., 0.1 meters per pixel).¹⁹ A geometric transformation matrix is constructed to map the global map coordinate system to the localized pixel coordinate system. All vector map elements retrieved from the map API must be translated relative to the ego-vehicle's origin, rotated mathematically according to the ego-vehicle's heading angle, and scaled inversely by the defined resolution.¹⁹

3.2 Rasterizing Polylines and Polygons without External Dependencies

A major architectural challenge in implementing pure functions for BEV rasterization is the pervasive historical reliance on external, C++ based graphics libraries. Standard implementations within deep learning data loaders depend heavily on OpenCV functions like `cv2.fillPoly` or `cv2.polylines` to perform the actual drawing.²⁰ To sever the dependency on OpenCV—a library known for complex state management and cross-platform compilation issues—and implement the geometry logic strictly within Python and NumPy, the fundamental computer graphics algorithms must be rebuilt from scratch utilizing vectorization.²¹

Graphic Operation	Source Geometry	Pure NumPy Implementation Strategy	Algorithmic Complexity (Per Item)
Line Discretization	Agent Trajectories, Lane Dividers	Vectorized Bresenham's Line Algorithm	$O(\max(\Delta X, \Delta Y))$
Polygon Rasterization	Drivable Surfaces, Vehicle Bounding Boxes	Bounding-Box Masking with Winding Number	$O(\text{Area}_{\text{bbox}} \cdot N_{\text{edges}})$

For lane lines and trajectory paths, mathematical polylines must be drawn onto the 2D pixel grid. Bresenham's line algorithm provides an efficient, integer-only approximation for determining the discrete pixels constituting a line segment between two arbitrary points (x_0, y_0) and (x_1, y_1) .²³ While iterative Python implementations of Bresenham's algorithm are prohibitively slow for millions of map segments, highly vectorized NumPy variants effectively parallelize the logic.²¹

The vectorized algorithmic logic evaluates the coordinate differentials Δx and Δy , determines the "driving axis" (the axis with the largest absolute change), and computes an error

accumulator to step along the subordinate axis.²⁵ In a pure NumPy paradigm, this is achieved by utilizing `np.linspace` to generate an array of points across the driving axis. The algorithm subsequently calculates the corresponding continuous coordinate values for the subordinate axis and utilizes `np.round` followed by casting to integers.²⁵ The resulting coordinate pairs are then used to index directly into the BEV tensor, modifying the values to indicate the presence of a line.

For solid geometric areas like drivable surfaces and vehicle bounding boxes, the task escalates to rasterizing filled polygons. Traditional computer graphics rely on scan-line fill algorithms, which operate by intersecting horizontal lines (scanlines) with all edges of the polygon.²⁶ The standard methodology requires finding the minimum and maximum Y coordinates, iterating through each integer Y in range, finding X intersections, sorting them, and filling the pixels between alternating pairs.²⁷

However, iterating through scanlines in Python is highly inefficient. A vastly superior, highly parallelized NumPy approach utilizes point-in-polygon logic based on localized bounding boxes. Given a set of polygon vertices, a minimum bounding box (aligned to the pixel grid) is computed. A grid of (X, Y) coordinates strictly for this localized sub-region is generated using `np.meshgrid`. A vectorized winding number or cross-product ray-casting algorithm checks all points in this sub-grid simultaneously against the polygon edges.²⁹ This calculation yields a boolean mask representing the interior of the polygon. This mask is then written directly into the corresponding sub-region of the BEV tensor utilizing standard NumPy slice assignments.³⁰ This approach maintains a pure functional boundary while executing orders of magnitude faster than iterative line-drawing.

3.3 Candidate Atom Specification: BEV Rasterization

Name: `rasterize_bev`

Description: Converts vector map geometries and agent history coordinates into a multi-channel top-down Bird's-Eye-View image tensor using purely vectorized grid filling and line drawing algorithms.

Source:

https://github.com/nutonomy/nuscenes-devkit/blob/master/python-sdk/nuscenes/map_expansion/map_api.py

License: Apache-2.0

Concept type: `data_assembly`

Signature: (agents: NDArray, map_elements: list, ego_pose: NDArray, raster_size: tuple[int,int], resolution: float) -> NDArray

Pure function boundary: Vector coordinates and raster constraints in; multi-channel discrete tensor out. Replaces external cv2 graphic dependencies with strict NumPy linear algebra logic and vectorized bounds checking.

Contracts:

- require: agents.ndim == 3
- require: raster_size > 0 and raster_size > 0
- require: resolution > 0.0
- ensure: output.shape == (num_channels, raster_size, raster_size)

Witness: An ego pose at the coordinate origin, a single straight lane polyline crossing the origin at a 45-degree angle, and one static rectangular agent. Assert that the resulting tensor correctly highlights the center pixel for the ego, an angled line of pixels matching the lane angle, and a small filled rectangular block for the agent.

Dependencies: numpy

CDG stages covered: lyft_motion_1st/bev_rasterization

4. Spatial Verification: RANSAC Homography Estimation

In multi-camera systems, sequential frame alignment tasks, or SLAM (Simultaneous Localization and Mapping) pipelines, establishing the geometric spatial correspondence between two 2D images is critical for robust perception.³¹ Keypoint matching algorithms (employing classical descriptors like SIFT or learning-based networks like SuperGlue) generate a set of putative point correspondences between two frames. However, these matches inherently contain a high proportion of false positives, known as outliers, resulting from repetitive environmental textures, physical occlusions, and dynamic objects moving independently of the camera.³¹

Spatial verification serves as the mechanism to sanitize these matches. It relies on estimating a geometric transformation—specifically, a planar homography matrix—that correctly aligns the true physical correspondences (inliers) while simultaneously rejecting the geometrically impossible outliers.³² The Random Sample Consensus (RANSAC) algorithm, paired with the Direct Linear Transform (DLT) mathematical formulation, acts as the industry standard mechanism for robust homography estimation under severe outlier contamination.³¹

4.1 Direct Linear Transform (DLT) Formulation

A homography H is a 3×3 transformation matrix that maps points from one projective plane to another: $x' = Hx$.³² Because the matrix operates on homogeneous coordinates (where the scale of the vector does not change the physical 2D location), the matrix is defined up to a non-zero scalar factor. Consequently, the 3×3 matrix possesses exactly 8 degrees of freedom.³⁴ Therefore, a minimum of 4 distinct, non-collinear point correspondences is strictly

required to solve for the elements of H .³⁴

For a given point correspondence $x_i = (X, Y, 1)^T$ in the source image and $x'_i = (x, y, 1)^T$ in the destination image, the relationship implies that the vectors x'_i and Hx_i are collinear. Their cross product $x'_i \times Hx_i = 0$ yields a system of equations.³⁴ Expanding this cross product produces two linearly independent equations for each point pair:

$$\begin{bmatrix} 0 & 0 & 0 & -X & -Y & -1 & yX & yY & y \\ X & Y & 1 & 0 & 0 & 0 & -xX & -xY & -x \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_9 \end{bmatrix} = 0$$

Given $N \geq 4$ correspondences, an overdetermined constraint matrix A of dimensions $2N \times 9$ is assembled by vertically stacking the equations for all point pairs.³⁴ The estimation problem resolves to finding the non-trivial vector h that satisfies $Ah = 0$. In practical applications involving sensor noise and discretization errors, an exact solution where $Ah = 0$ rarely exists. Thus, the formulation pivots to an optimization problem: seek the total least squares solution that minimizes the norm $\|Ah\|$ subject to the necessary unit norm constraint $\|h\| = 1$ to prevent the trivial zero solution.³⁵

The Direct Linear Transform (DLT) constructs a constraint matrix A of size $2N \times 9$ from the N point correspondences. To find the optimal homography vector, the algorithm relies on Singular Value Decomposition (SVD), which factors the matrix A into the product of three matrices: $A = U\Sigma V^T$. In this decomposition, U and V^T are orthogonal matrices, and Σ contains the singular values. The unit vector that minimizes the reprojection error corresponds to the right singular vector associated with the smallest singular value. Programmatically, this is extracted as the last row of the V^T matrix. This 9×1 vector h is then reshaped into the 3×3 Homography matrix H . In a pure NumPy environment, this operation is elegantly and efficiently executed utilizing the `np.linalg.svd(A)` function.³⁴

4.2 RANSAC Execution Loop

Because a single outlier mixed into the minimal 4-point sample will completely corrupt the DLT estimation, the system cannot simply feed all putative matches into the SVD solver. RANSAC acts as a robust wrapper, iteratively testing minimal hypotheses to find a geometric

consensus.³¹ The theoretical foundation of RANSAC provides probabilistic guarantees; the required number of iterations N to ensure a high probability p of selecting at least one outlier-free sample of size s is governed by the equation

$$N = \log(1 - p) / \log(1 - (1 - e)^s), \text{ where } e \text{ is the estimated outlier ratio.}$$

The pure-function RANSAC iteration follows a strict, stateless progression in NumPy³⁹:

1. **Hypothesis Generation:** A random seed controls the sampling logic to maintain purity and determinism across testing suites. Using a local instance of `np.random.RandomState(seed)`, the algorithm selects 4 random, distinct point pairs from the putative matches.³¹
2. **Model Estimation:** The algorithm executes the 4-point DLT formulation (assembling the 8×9 constraint matrix and calling SVD) to compute a temporary hypothesis homography, H_{temp} .⁴⁰
3. **Model Verification:** The temporary matrix H_{temp} is applied simultaneously to all N source points.³⁹ The source points are converted to homogeneous coordinates $(X, Y, 1)$, multiplied by H_{temp} using `np.dot`, and then divided by the resulting third coordinate vector to perform the required perspective normalization.³⁹
4. **Consensus Scoring:** The algorithm calculates the $L2$ geometric distance between the newly projected source points and the actual observed destination points. A boolean array identifies the "inliers" where this reprojection distance falls below a pre-defined scalar threshold parameter.⁴⁰
5. **Iteration Control:** If the current sum of the inlier mask exceeds the previous best iteration's consensus count, the algorithm stores H_{temp} and the corresponding inlier mask as the reigning champion.⁴⁰ This loop repeats strictly until `max_iterations` is reached.
6. **Refinement:** While the iteration champion H_{temp} was generated using only 4 points, optimal accuracy requires utilizing all available verified data. The algorithm recomputes a final H utilizing the DLT procedure on every point pair flagged by the best inlier mask, thereby minimizing the global total least squares error over the entire consensus set.³¹

This methodology strictly relies on primitive `numpy.linalg` operations, purposefully evading the complex internal C++ states, unexposed memory allocations, and implicit multi-threading behaviors associated with standard library calls like `cv2.findHomography`.⁴¹

4.3 Candidate Atom Specification: RANSAC Homography

Name: ransac_homography

Description: Estimates a robust 3x3 homography matrix from putative 2D point correspondences with iterative outlier rejection using the Direct Linear Transform (DLT) and RANSAC algorithms.

Source: <https://github.com/towardsautonomy/homography>

License: MIT

Concept type: geometry

Signature: (src_points: NDArray, dst_points: NDArray, threshold: float, max_iterations: int, seed: int) -> tuple

Pure function boundary: Arrays of matched coordinates in, 3x3 transformation matrix and boolean inlier mask out. Pseudo-random sampling relies strictly on the explicitly passed local seed to guarantee deterministic execution.

Contracts:

- require: src_points.ndim == 2 and src_points.shape == 2
- require: dst_points.shape == src_points.shape
- require: src_points.shape >= 4
- require: threshold > 0.0 and max_iterations > 0
- ensure: homography_matrix.shape == (3, 3)
- ensure: inlier_mask.shape == (src_points.shape,) and inlier_mask.dtype == bool

Witness: A synthetic set of 10 points shifted by a known translation and rotation (forming a perfect homography) accompanied by 3 severe outlier points. Run the function and verify that the output homography correctly maps the true points while the inlier mask correctly flags the 3 outliers as False.

Dependencies: numpy

CDG stages covered: facebook_image_similarity_1st/spatial_verification

Conclusion

The successful engineering of multi-modal 3D perception pipelines, particularly those destined for competitive evaluation or high-stakes autonomous driving deployment, relies entirely on the structural integrity of the underlying geometric and data-assembly operations. Shifting from highly integrated, stateful, or hardware-bound implementations (such as OpenCV routines, PyTorch-specific CUDA kernels, or deep-framework idiosyncrasies) towards pure, statically typed NumPy functions offers profound architectural benefits. The candidate primitives proposed in this report—hard voxelization executed via efficient hash indexing, geometrically correct multi-modal point painting via vectorized bilinear sampling, vector-to-tensor rasterization unconstrained by external graphic libraries, and SVD-driven spatial verification—collectively guarantee mathematically deterministic execution. By adhering strictly to immutable function boundaries and enforcing explicit operational contracts, these

components serve as the foundational scaffolding necessary for building resilient, highly debuggable, and vastly extensible autonomous driving architectures.

Works cited

1. This repository is an open-source PointPainting package which is easy to understand, deploy and run! - GitHub, accessed May 4, 2026, <https://github.com/Song-Jingyu/PointPainting>
2. nusenes-devkit/python-sdk/nusenes/utls/geometry_utils.py at ..., accessed May 4, 2026, https://github.com/nutonomy/nusenes-devkit/blob/master/python-sdk/nusenes/utls/geometry_utils.py
3. Point Cloud Voxelization with Python (numpy & scipy) | by Shaig Hamzaliyev | Medium, accessed May 4, 2026, <https://medium.com/@h.shaig93/voxelization-with-python-numpy-scipy-b7f013890da6>
4. Source code for mmdet3d.core.voxel.voxel_generator - MMDetection3D's documentation!, accessed May 4, 2026, https://mmdetection3d.readthedocs.io/en/v1.0.0rc0/_modules/mmdet3d/core/voxel/voxel_generator.html
5. Point2Voxel generated coordinates out of range? · Issue #1589 · open-mmlab/OpenPCDet, accessed May 4, 2026, <https://github.com/open-mmlab/OpenPCDet/issues/1589>
6. can it run without GPU? · Issue #181 · open-mmlab/OpenPCDet · GitHub, accessed May 4, 2026, <https://github.com/open-mmlab/OpenPCDet/issues/181>
7. danielhavir/voxelize3d: 3D Point Cloud Voxelization in NumPy - GitHub, accessed May 4, 2026, <https://github.com/danielhavir/voxelize3d>
8. nuScenes, accessed May 4, 2026, <https://www.nuscenes.org/nuscenes?tutorial=prediction>
9. nuScenes devkit tutorial, accessed May 4, 2026, https://www.nuscenes.org/tutorials/nuscenes_tutorial.html
10. [PDF] PointPainting: Sequential Fusion for 3D Object Detection - Semantic Scholar, accessed May 4, 2026, <https://www.semanticscholar.org/paper/PointPainting%3A-Sequential-Fusion-for-3D-Object-Vora-Lang/8648f59b62dda3900d597e944abe8af51a18f665>
11. PointPainting: Sequential Fusion for 3D Object Detection - CVF Open Access, accessed May 4, 2026, https://openaccess.thecvf.com/content_CVPR_2020/papers/Vora_PointPainting_Sequential_Fusion_for_3D_Object_Detection_CVPR_2020_paper.pdf
12. undefinedzero/PointPainter: PointPainted 3D detectors - GitHub, accessed May 4, 2026, <https://github.com/undefinedzero/PointPainter>
13. Image Resampling Using Bilinear Interpolation in Python (NumPy & SciPy) - GitHub, accessed May 4, 2026, <https://github.com/alivcor/Image-Unsampling>
14. Simple, efficient bilinear interpolation of images in numpy and python - Stack Overflow, accessed May 4, 2026,

- <https://stackoverflow.com/questions/12729228/simple-efficient-bilinear-interpolation-of-images-in-numpy-and-python>
15. Implementing Bilinear Interpolation for Image Resizing | by meghal darji - Medium, accessed May 4, 2026,
<https://meghal-darji.medium.com/implementing-bilinear-interpolation-for-image-resizing-357cbb2c2722>
 16. l5kit/examples/visualisation/visualise_data.ipynb at main - GitHub, accessed May 4, 2026,
https://github.com/woven-planet/l5kit/blob/master/examples/visualisation/visualise_data.ipynb
 17. Scaling Out Motion Prediction for Autonomous Vehicles with L5Kit, Ray, and Wandb - Wandb, accessed May 4, 2026,
<https://wandb.ai/l5-demo/l5-common/reports/Scaling-Out-Motion-Prediction-for-Autonomous-Vehicles-with-L5Kit-Ray-and-W-B---VmlldzoyNTMwODI4>
 18. PolyMap: Generating High Definition Map based on Rasterized Polygons - arXiv, accessed May 4, 2026, <https://arxiv.org/html/2511.05944v1>
 19. nuscenet-devkit/python-sdk/nuscenet/map_expansion/map_api.py ..., accessed May 4, 2026,
https://github.com/nutonomy/nuscenet-devkit/blob/master/python-sdk/nuscenet/map_expansion/map_api.py
 20. Draw a filled polygon using the OpenCV function fillPoly() - GeeksforGeeks, accessed May 4, 2026,
<https://www.geeksforgeeks.org/python/draw-a-filled-polygon-using-the-opencv-function-fillpoly/>
 21. Directly "plot" line segments to numpy array - Stack Overflow, accessed May 4, 2026,
<https://stackoverflow.com/questions/33466231/directly-plot-line-segments-to-numpy-array>
 22. How to draw and fill a polygon on a grid array using pure Numpy functions? - Stack Overflow, accessed May 4, 2026,
<https://stackoverflow.com/questions/20897140/how-to-draw-and-fill-a-polygon-on-a-grid-array-using-pure-numpy-functions>
 23. A simple implementation of Bresenham's line drawing algorithm - GitHub, accessed May 4, 2026, <https://github.com/encukou/bresenham>
 24. Bitmap/Bresenham's line algorithm - Rosetta Code, accessed May 4, 2026,
https://rosettacode.org/wiki/Bitmap/Bresenham%27s_line_algorithm
 25. Bresenham's line algorithm in n-dimensions « Python recipes « ActiveState Code, accessed May 4, 2026,
<https://code.activestate.com/recipes/578112-bresenham-s-line-algorithm-in-n-dimensions/>
 26. Scanline Rasterization - Gabor Makes Games, accessed May 4, 2026,
https://gabormakesgames.com/blog/polygons_scanline.html
 27. Graphics program to implement polygon filling using scan line algorithm, accessed May 4, 2026,
<https://theprogrammingworld.wordpress.com/2016/03/13/graphics-program-to-i>

- [mplement-polygon-filling-using-scan-line-algorithm/](#)
28. simple implementation of the scan-line polygon fill algorithm for arduino (allows concave sections, but not intersecting edges) - GitHub Gist, accessed May 4, 2026, <https://gist.github.com/robey/08ddfe9f653156606adb56094d89d7f3>
 29. Rasterizing polygon from numpy arrays - main@rasterio.groups.io, accessed May 4, 2026, https://rasterio.groups.io/g/main/topic/rasterizing_polygon_from/78188800
 30. Rasterize Vectors w. Rasterio - PyGIS, accessed May 4, 2026, https://pygis.io/docs/e_raster_rasterize.html
 31. A homography estimation project using a RANSAC algorithm. - GitHub, accessed May 4, 2026, <https://github.com/dastratakos/Homography-Estimation>
 32. Automatic Computation of a Homography by RANSAC Algorithm - Purdue Engineering, accessed May 4, 2026, https://engineering.purdue.edu/kak/computervision/ECE661.08/solution/hw4_s1.pdf
 33. Step-by-Step Guide to Using RANSAC in OpenCV using Python - GeeksforGeeks, accessed May 4, 2026, <https://www.geeksforgeeks.org/python/step-by-step-guide-to-using-ransac-in-opencv-using-python/>
 34. Estimating the Homography Matrix with the Direct Linear Transform (DLT) - Medium, accessed May 4, 2026, <https://medium.com/@insight-in-plain-sight/estimating-the-homography-matrix-with-the-direct-linear-transform-dlt-ec6bbb82ee2b>
 35. Direct Linear Transform - Carnegie Mellon University, accessed May 4, 2026, https://www.cs.cmu.edu/~16385/s17/Slides/10.2_2D_Alignment_DLT.pdf
 36. Singular Value Decomposition (SVD) with NumPy | CodeSignal Learn, accessed May 4, 2026, <https://codesignal.com/learn/courses/eigenvalues-eigenvectors-and-diagonalization-with-numpy/lessons/singular-value-decomposition-svd-with-numpy>
 37. Homography Matrix Estimation using SVD - GitHub, accessed May 4, 2026, <https://github.com/towardsautonomy/homography>
 38. How to calculate homography matrix with DLT and python? : r/computervision - Reddit, accessed May 4, 2026, https://www.reddit.com/r/computervision/comments/2h1yff/how_to_calculate_homography_matrix_with_dlt_and/
 39. Assignment 4: RANSAC and Panorama Sticking - UBC Computer Science, accessed May 4, 2026, <https://www.cs.ubc.ca/~kmyi/teaching/cpsc425/assignment4.html>
 40. Lecture 13 Homographies and RANSAC - mit csail, accessed May 4, 2026, <http://6.869.csail.mit.edu/fa12/lectures/lecture13ransac/lecture13ransac.pdf>
 41. arXiv:2302.09997v1 [cs.CV] 20 Feb 2023, accessed May 4, 2026, <https://arxiv.org/pdf/2302.09997>